

These 15 questions will take you from **basic Pandas/NumPy data cleaning** → **missing values** → **renaming columns** → **changing data types** → **aggregation** → **grouping** → **multiple aggregate functions** → **merging/joining DataFrames** → **merging using key columns**, with practical and debugging problems.

Q1. — Easy

Given:

```
df = pd.DataFrame({
    "name": ["Aman", "Priya", "Rahul", "Neha"],
    "marks": [75, None, 82, None]
})
```

Find out which values are missing in the DataFrame.

Then find the **total number of missing values**.

Q2. — Easy

Using the same DataFrame:

```
df = pd.DataFrame({
    "name": ["Aman", "Priya", "Rahul", "Neha"],
    "marks": [75, None, 82, None]
})
```

Create a new version of the DataFrame where missing **marks** values are replaced with 0.

Print the result.

Think about why replacing missing values might be useful before performing calculations.

Q3. — Easy

Given:

```
df = pd.DataFrame({
    "student_name": ["Aman", "Priya", "Rahul"],
```

```
"student_marks": [75, 88, 92],  
"student_age": [17, 18, 17]  
})
```

Rename the columns so they become:

```
name  
marks  
age
```

Print the DataFrame.

Q4. — Medium

Given:

```
df = pd.DataFrame({  
    "name": ["Aman", "Priya", "Rahul"],  
    "age": ["17", "18", "17"],  
    "marks": ["75", "88", "92"]  
})
```

Check the data types of the columns.

Then change **age** and **marks** from strings to an appropriate numeric datatype.

Check the data types again.

Q5. — Medium

Given:

```
df = pd.DataFrame({  
    "name": ["Aman", "Priya", "Rahul", "Neha"],  
    "marks": [75, 88, 92, 65]  
})
```

Calculate:

- total marks
- average marks

- highest marks
- lowest marks

Use Pandas aggregation rather than manually calculating the values.

Q6. — Medium

Given:

```
df = pd.DataFrame({  
    "department": ["IT", "HR", "IT", "Sales", "HR", "IT"],  
    "salary": [25000, 30000, 28000, 35000, 32000, 27000]  
})
```

Group the employees by department.

For each department, find the **average salary**.

Then find the **number of employees** in each department.

Q7. — Application

A shop has this sales data:

```
sales = pd.DataFrame({  
    "product": ["Pen", "Pen", "Bag", "Bag", "Notebook", "Notebook"],  
    "quantity": [10, 20, 2, 3, 5, 8],  
    "price": [20, 20, 500, 500, 80, 80]  
})
```

The shop owner wants to know **how many units of each product were sold in total**.

Find the answer using the DataFrame.

Q8. — Application

Using the same `sales` DataFrame, the shop owner now wants to know for each product:

- total quantity sold
- average price
- highest price

Produce one grouped result containing all three pieces of information.

Think about how you can apply **multiple aggregation functions** to grouped data.

Q9. — Application

You receive two DataFrames:

```
customers = pd.DataFrame({
    "customer_id": [101, 102, 103, 104],
    "name": ["Aman", "Priya", "Rahul", "Neha"]
})
```

```
orders = pd.DataFrame({
    "customer_id": [101, 102, 101, 103],
    "amount": [500, 800, 300, 1000]
})
```

You want to connect each order with the corresponding customer's name.

Create a combined DataFrame containing the customer name and order amount.

The common column is `customer_id`.

Q10. — Tricky

Consider:

```
df = pd.DataFrame({
    "name": ["Aman", "Priya", "Rahul", "Neha"],
    "marks": [70, None, 80, None]
})
```

Predict what happens to the average marks if you calculate the average directly while the DataFrame still contains missing values.

Then compare that with replacing the missing values with 0 before calculating the average.

Why are the two averages different?

The important part is not just getting the numbers — explain what the missing values mean in each situation.

Q11. — Tricky — Find the mistake

A beginner has this DataFrame:

```
df = pd.DataFrame({
    "name": ["Aman", "Priya", "Rahul"],
    "age": ["17", "18", "17"]
})
```

They write:

```
df["age"].astype(int)
print(df)
```

But they expect the DataFrame's `age` column to permanently become integers.

Find the mistake in their thinking.

Explain what happens to the result of `astype()` and what needs to be done if they want the DataFrame itself to contain the changed datatype.

Q12. — Tricky — Understand merging

Predict what the merged result should contain:

```
students = pd.DataFrame({
    "id": [1, 2, 3],
    "name": ["Aman", "Priya", "Rahul"]
})
```

```
marks = pd.DataFrame({
    "id": [1, 2, 4],
    "marks": [75, 88, 91]
})
```

Merge these DataFrames using `id` as the key.

Pay special attention to:

- Which students have matching records?
 - What happens to student `3`?
 - What happens to `id = 4`?
 - Why does choosing the type of merge matter?
-

Q13. — Mixed

Write a function that receives a sales DataFrame containing:

```
product  
category  
quantity  
price
```

The function should create a new column representing the **total value of each sale**.

Then group the data by `category` and calculate the total sales value for each category.

Test your function with your own small DataFrame.

Q14. — Mixed

You have customer information in one DataFrame and purchase information in another:

```
customers = pd.DataFrame(  
    "customer_id": [1, 2, 3, 4],  
    "name": ["Aman", "Priya", "Rahul", "Neha"],  
    "city": ["Delhi", "Mumbai", "Delhi", "Pune"]  
)
```

```
purchases = pd.DataFrame(  
    "customer_id": [1, 1, 2, 3, 3],  
    "amount": [500, 200, 800, 300, 700]  
)
```

Create a small analysis that determines:

- each customer's total purchase amount
- the city-wise total purchase amount
- the customer who spent the most

Think about the order in which you should **merge, group, and aggregate** the data.

Q15. — Mastery Challenge

You are given three small datasets:

```
customers = pd.DataFrame({
    "customer_id": [101, 102, 103, 104],
    "name": ["Aman", "Priya", "Rahul", "Neha"],
    "city": ["Delhi", "Mumbai", "Delhi", "Pune"]
})
```

```
orders = pd.DataFrame({
    "order_id": [1, 2, 3, 4, 5],
    "customer_id": [101, 102, 101, 103, 104],
    "amount": [500, 800, 300, 1000, 600]
})
```

```
extra = pd.DataFrame({
    "customer_id": [101, 102, 103, 104],
    "age": ["17", "18", "17", "19"]
})
```

Perform a small data-cleaning and analysis task:

- combine the customer information with the extra customer information
- make sure **age** has an appropriate numeric datatype
- connect the customer information with their orders
- find the total amount spent by each customer
- find the average, highest, and lowest order amount for each customer
- identify which city has the highest total sales

You should decide yourself **where missing-value handling, datatype conversion, merging, grouping, and multiple aggregation functions are useful**.

Mastery Check

After completing these 15 questions, you should be able to:

1. **Detect and handle missing values** appropriately instead of blindly replacing them.
2. **Rename columns and change datatypes** when cleaning a DataFrame.
3. **Group and aggregate data** to answer questions such as totals, averages, counts, minimums, and maximums.
4. **Apply multiple aggregation functions** to grouped data.
5. **Merge/join DataFrames using key columns** and understand why different merge types produce different results.